A Benchmark Environment for Offline Reinforcement Learning in Racing Games

Girolamo Macaluso[1], Alessandro Sestini[2], and Andrew D. Bagdanov[1]
[1]*University of Florence*, [2]*SEED - Electronic Arts (EA)*
{girolamo.macaluso, andrew.bagdanov}@unifi.it, asestini@ea.com

2024 IEEE Conference on Games (CoG) | 979-8-3503-5067-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/CoG60054.2024.10645657

***Abstract*—Offline Reinforcement Learning (ORL) is a promising approach to reduce the high sample complexity of traditional Reinforcement Learning (RL) by eliminating the need for continuous environmental interactions. ORL exploits a dataset of pre-collected transitions and thus expands the range of application of RL to tasks in which the excessive environment queries increase training time and decrease efficiency, such as in modern AAA games. This paper introduces OfflineMania a novel environment for ORL research. It is inspired by the iconic TrackMania series and developed using the Unity 3D game engine. The environment simulates a single-agent racing game in which the objective is to complete the track through optimal navigation. We provide a variety of datasets to assess ORL performance. These datasets, created from policies of varying ability and in different sizes, aim to offer a challenging testbed for algorithm development and evaluation. We further establish a set of baselines for a range of Online RL, ORL, and hybrid Offline to Online RL approaches using our environment.**

I. Introduction

Reinforcement Learning (RL) has become increasingly popular in the gaming industry as it offers a promising way of creating immersive gaming experiences. From training AI-controlled non-player characters [1, 2] to automated game testing [3, 4]. However the widespread use of RL is often limited by its sample complexity which makes training in complex environments, such as modern AAA games, slow and inefficient. Offline RL (ORL) has recently garnered interest as a framework aimed at improving the sample efficiency of RL agents [5]. With ORL, one can completely eliminate the need for interaction with the environment and instead rely on a previously collected dataset of experiences. Such datasets can be made readily accessible to game developers; for instance, they could use samples obtained from playtesting sessions or data extracted from previously released games.

In this paper we introduce OfflineMania a novel game environment for Online RL and ORL, centered around a single-agent racing game inspired by the iconic *TrackMania* [6] game series. Our environment, built using the Unity 3D game engine [7], provides a track in which the agent must complete the race through optimal navigation. Moreover, we provide datasets of agent experiences tailored specifically for benchmarking ORL techniques. These datasets are of varying quality, ranging from those generated using random policies to those crafted by expert agents. We additionally offer smaller and mixed versions of these datasets. These variants are designed to test algorithmic performance under complex scenarios that challenge the robustness of learning methods. Our work focuses on providing a gaming testbed environment *and* multiple datasets tailored for game AI research in ORL. To the best of our knowledge such a combination is not currently available in the existing literature. We further provide a study assessing the performance of different Online RL and ORL algorithms using our new datasets. We also investigate the performance of fine-tuning policies trained offline using Online RL. This last framework is a more natural approach in game development, as it allows developers to take advantage of datasets to create a policy that then can be effectively improved with fewer game interactions.

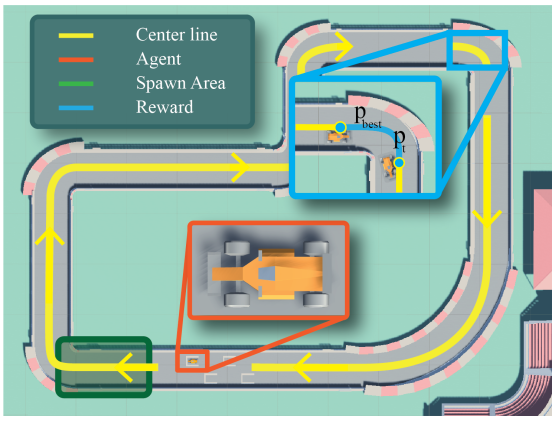

Fig. 1. Visualization of OfflineMania, highlighting the track centerline used in the reward function, the episode starting area, the agent, and the positive part of the reward during a transition.

The key contributions of this work are:

- we introduce OfflineMania, a new environment inspired by TrackMania developed using the Unity 3D engine;
- we provide diverse datasets of varying sizes, collected using policies of different expertise levels; and
- we present results for a variety of baseline algorithms including both Online and ORL approaches, as well as hybrid methods that combine Offline training with Online fine-tuning.

II. Related work

Offline Reinforcement Learning (ORL). Offline Reinforcement Learning is a promising way to address the sample

979-8-3503-5067-8/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: Comenius University. Downloaded on March 05,2025 at 09:17:18 UTC from IEEE Xplore. Restrictions apply.

complexity challenges faced by Online RL. The core objective of ORL is to create a robust policy from a fixed dataset of pre-collected environment transitions, without requiring further online interactions with the environment [8, 9, 10]. ORL has emerged as a promising approach to training game agents [11]. Modern AAA games are often computationally demanding, slow to simulate, and inherently unstable, all of which lead to increased need for extensive interactions with the environment.

ORL has significantly benefited from the development of benchmarks like D4RL [12], which provides a wide range of datasets across various domains such as locomotion, robotic manipulation, and vision-based autonomous driving. However, to the best of our knowledge, there is no ORL dataset available in the literature specifically tailored for studying these techniques in gaming environments, particularly in the context of racing games. This paper aims to bridge this gap.

Offline to Online RL. The Offline to Online approach focuses on how to effectively *improve* policies trained with ORL in a online setting which allows further environment interactions. It is a promising approach for training game agents [13] since a game in development can change on a daily basis, potentially rendering the datasets collected in an environment iteration insufficient for subsequent iterations. Offline to Online RL can allow game developers to seamlessly transition to new environment with minimal online interactions. However, this transition poses significant challenges [14, 15], including dealing with distribution shifts between the offline data and the new online interactions. For these reasons, with this paper we aim to provide a benchmark suite for investigating the impact of Offline to Online training in gaming environments.

III. Environment and Datasets

A. Environment

We developed OfflineMania, shown in Figure 1, using the Unity 3D game engine leveraging the ML-Agents package [7]. It features a Gymnasium-compatible interface [16], ensuring straightforward integration into existing experimental setups. The game is computationally efficient, with the game speed easily adjustable to speed up online training process. Additionally, the environment supports rendering capabilities, offering a birds-eye view of the car. This visualization facilitates qualitative assessment of agent behavior and simplifies the evaluation process.

We now describe the main elements of the environment: the state and action-space, the reward function, and episode loop.

State Space. The state space is a vector in $\mathcal{R}^{33}$. It is composed by 15 raycasts covering a 180-degree field of view in front of the car, with each ray are associated two values: one indicating the presence of an object within its path, and the other specifying the distance to the detected object. Additionally, the components of the velocity of the car are included as part of the state representation.

Action Space. The agent action space consists of two continuous values. The first value controls the steering angle of the car which ranges from -1 (indicating a left turn) to 1 (indicating a right turn). The second value controls the acceleration or braking of the car, with a value of 1 corresponding to full acceleration and a value of -1 representing braking or reversing when the car is stationary.

Reward Signal. Our reward function draws inspiration from prior work [2]. We denote with p_t the position of the car projected onto the track centerline at timestep t, and by p_{best} the most advanced position achieved thus far in the episode. Our reward is then:

$$r_t = r_t^{\text{prog}} - \begin{cases} \lambda \parallel v_{\text{car}} \parallel & \text{if in contact with wall} \\ 0 & \text{otherwise} \end{cases}, \quad (1)$$

where r_t^{prog} quantifies the progress of position p_t along the centerline relative to p_{best}, as shown in Figure 1. We set r_t^{prog} to 0 if p_t does not advance beyond p_{best}. v_{car} is the magnitude of the velocity at the moment of impact, and λ is a fixed coefficient penalizing collisions. In our environment, $\lambda = 50$.

Episode. During each episode the vehicle starts with its position dynamically chosen within a designated square area before the fist turn in the track. We also randomize the orientation of the car, between -30 and 30 degrees from the centerline, which ensures that it is always facing the correct direction. Every episode has a fixed length of 2,000 steps. In this many steps an expert agent can complete at most 5 laps.

B. Datasets

In order to support comprehensive research in ORL, we generated a diverse series datasets. First, we train three distinct policies using Proximal Policy Optimization (PPO) [17], stopping the training after 1,000, 5,000, and 12,500 network updates, respectively. Each of the trained policy represents varying degrees of ability in navigating the race track. The policies obtain mean cumulative reward of -360, 327, and 1183 respectively, over five episodes. The first policy struggles with the initial corner. The second policy, while capable of occasionally completing the track, exhibits inconsistent performance. In contrast, the third policy consistently achieved high performance by efficiently navigating the track, including corner-cutting strategies, successfully completing 5 laps in each episode.

Using these three policies, we collected three distinct datasets: *basic*, *medium*, and *expert*, each consisting of 100,000 transitions. Additionally, we created mixed datasets, one consisting of 200,000 transitions in total and another of only 5,000. We refer to these datasets with *mix large* and *mix small*, respectively. The mixed datasets consist of 90% transitions sampled from the basic policy, 7% from the medium policy, and only 3% from the expert policy. The distribution of transitions in the mixed datasets was chosen to simulate a complex scenario in which ORL algorithms must stitch together different behaviors in order to correctly learn an optimal policy. The smaller version of the mixed dataset is useful to understand the behavior of ORL approaches when dealing with small datasets. Following this idea, we built

Authorized licensed use limited to: Comenius University. Downloaded on March 05,2025 at 09:17:18 UTC from IEEE Xplore. Restrictions apply.

another datasets from only 5,000 transitions collected exclusively from the basic policy, called *basic small*. Those two variants – *mix small* and *basic small* – offer a more demanding testbed for evaluating the robustness and adaptability of ORL algorithms under complex training conditions.

IV. Benchmark Study

OfflineMania aims to be a testbed for developing new training techniques. We provide results of a set of baselines for widely recognized methods for Online RL, ORL, and Offline to Online RL. For all approaches we present the mean results over five different seeds. All experiments were conducted using a system equipped with a Nvidia RTX 2070 and an AMD Ryzen 3600X processor.

For Online RL baselines, we opted for two state-of-the-art methods: Proximal Policy Optimization (PPO) [17] and Soft Actor Critic (SAC) [18]. PPO, known for its efficacy and robustness to hyperparameter selection, represents a widely used policy-based approach. Similarly, we selected SAC, an actor-critic algorithm, for its efficiency and for its importance in the RL landscape. We trained our PPO agent over 12,500 network updates, for a total of 15 million environment interactions and approximately 10 hours of training time. In contrast, training with SAC spanned 3 million network updates, with an equivalent number of environment interactions, totaling approximately 20 hours of training time.

For ORL approaches, we chose Conservative Q-Learning (CQL) [10], Twin Delayed Deep Deterministic policy gradient with Behavioral cloning (TD3BC) [9], and Implicit Q-Learning (IQL) [8]. For all algorithms we present results after 300,000 network updates, corresponding to about one hour of training time for all algorithms.

For Offline to Online approaches, we compare various methods. These include: an approach combining TD3BC [9] for offline training and TD3 [19] for online fine tuning; IQL [8], following the fine-tuning process outlined in the original paper; Jump Start Reinforcement Learning (JSRL) [20], which utilizes offline policies as guides for online training; Policy EXpainsion (PEX) [21], a method combining offline policies with online training to enhance exploration; and the work of Macaluso et al. [22] that we will refer as SDBG, designed specifically for small offline datasets, utilizing a world model based augmentation to improve offline training. For each approach we present results after 300,000 offline network updates and 1 million online fine-tuning updates, for a total of about 4 hours of training across all algorithms. Since PEX, SDGB, and JSRL are agnostic to the algorithm used, we decided to show results using IQL.

A. Online RL Results

We use online RL for training a policy from scratch with environment interactions. The resulting mean reward achieved by PPO at the end of the training was 1183, indicative of consistent high-quality behaviors on the track. The policy is proficient at navigating the track, and is also able to cut corners effectively. It can complete a lap in 385 steps, for a total of 5 laps in a single episode. PPO succeeds at the cost of a large number of environment interactions (about 15 million).

Conversely, despite training for 20 hours and 3 million environment interactions, the SAC outcomes are considerably less promising. The SAC policy achieves a total mean reward of only 215 and demonstrates suboptimal performance characterized by slower and less stable navigation of the track compared to the PPO-trained policy. This highlights the limitations of SAC, despite its better sample efficiency, in producing effective policies even after extensive training.

These results quantify the efficacy of Online RL approaches. However, they also highlight the challenges posed by the slow training speed and significant sample inefficiency.

TABLE I
Average Rewards of Offline Reinforcement Learning Training after 300,000 Network Updates.

Methods	TD3BC	CQL	IQL
Expert	-3981±57	-3325±1353	**1192±1**
Medium	335±87	-4227±571	**789±58**
Basic	12±9	39±81	**98±38**
Mix Large	219±96	-4080±873	**828±38**
Mix Small	-1125±154	-3972±858	**10±32**
Basic Small	**64±31**	-3488±626	20±102

B. Offline RL Results

In Table I we present the results for ORL baselines on the datasets described in Section III-B. For all algorithms the training time for 300,000 network updates was under one hour. Remarkably, IQL consistently outperforms TD3BC and CQL across nearly all datasets. Of particular note is the ability of IQL to learn policies from the expert dataset that even surpass the performance of the policy used to generate it. On all other datasets, IQL produces policies that are capable of navigating the track without major collisions.

Conversely, TD3BC and CQL unexpectedly fall short on all datasets, most notably on the expert dataset. This may be attributed to sensitivity to the hyperparameters that is a well known problem in ORL [5].

C. Offline to Online RL Results

In this section we report baseline performance for the combination of ORL pre-training and Online RL fine-tuning. Such approaches are useful when an offline policy fails to meet deployment standards. In these cases, we would like to improve the offline-trained policy by allowing some interactions with the environment, while still minimizing such interactions.

While this process might appear straightforward, it is complex primarily due to the distributional shift problem. Distributional shift occurs during the first training iterations when moving to online training, as the agent navigates into unexplored state-action spaces. In this setting the values of the Q-function trained during the offline phase may become highly inaccurate. This inaccuracy can lead to incorrect policy evaluations and arbitrary policy updates in these unseen states, which undermines the policy learned through ORL [15].

Authorized licensed use limited to: Comenius University. Downloaded on March 05,2025 at 09:17:18 UTC from IEEE Xplore. Restrictions apply.

TABLE II
AVERAGE REWARDS FOR OFFLINE-TO-ONLINE APPROACHES AFTER 300,000 OFFLINE NETWORK UPDATES AND 1 MILLION ONLINE UPDATES.

Methods	TD3BC+TD3	IQL	SDBG-IQL	PEX-IQL	JSRL-IQL
Expert	15±11	1192±4	**1199±3**	-2425±2132	1193±1
Medium	-497±773	856±58	852±12	776±39	**856±1**
Basic	11±7	71±13	**130±35**	-173±504	99±33
Mix Large	-2538±1170	932±50	**1152±12**	671±175	1142±2
Mix Small	-3886±923	-614±882	**340±187**	-2147±1561	-826±86
Basic Small	-4352±1425	3±33	105±3	-2200±2042	**152±1**

In Table II we present the results of the experiments in this setup. The combination of TD3BC and TD3 fails to improve the offline policy and achieves worse performances on all datasets. This might be due to the distributional shift problem which is not explicitly addressed by this approach. On the other hand, IQL demonstrates good fine-tuning performance and improves its offline results in almost all datasets. Only for the *mix small* dataset the score is substantially reduced. This is probably due to the difficulty in learning with such a small dataset derived from different policies.

Although PEX attempts to directly address the challenge of training an online policy after offline pre-training, it achieves the worst performance – lower even than the standard fine-tuning process of IQL. On the other hand, SDBG and JSRL show remarkable performance. Even though SDBG specifically addresses training setups with small datasets – as seen in the *mix small* and *basic small* results from Table II – it is the only algorithm that improves the offline performance in *all* tasks. JSRL achieves good performance across all tasks as well, except again for the *mix small* dataset.

V. DISCUSSION

ORL is a promising approach to fostering more widespread use of RL in modern video games, however there is a lack of environments with datasets that can be used to test the capabilities of ORL methods in challenging conditions. This paper introduces a novel game environment which serves as a benchmark for ORL research. We provide datasets of varying data quality and quantity which facilitates the simulation of complex training scenarios[1]. Additionally, we present results from state-of-the-art methods for Online RL, ORL, and hybrid Offline to Online RL approaches. This benchmark aims to facilitate future investigation into the use of offline data in gaming environments. By leveraging offline datasets, we can effectively mitigate the challenges in applying RL techniques to modern games, thereby contributing the integration of RL into modern game development workflows.

REFERENCES

[1] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot *et al.*, "Mastering the game of go with deep neural networks and tree search," *nature*, 2016.

[2] F. Fuchs, Y. Song, E. Kaufmann, D. Scaramuzza, and P. Dürr, "Super-human performance in gran turismo sport using deep reinforcement learning," *IEEE Robotics and Automation Letters*, 2021.

[3] A. Sestini, L. Gisslén, J. Bergdahl, K. Tollmar, and A. D. Bagdanov, "Automated gameplay testing and validation with curiosity-conditioned proximal trajectories," *IEEE Transactions on Games*, 2022.

[4] J. Bergdahl, C. Gordillo, K. Tollmar, and L. Gisslén, "Augmenting automated game testing with deep reinforcement learning," in *IEEE Conference on Games (CoG)*, 2020.

[5] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline reinforcement learning: Tutorial, review, and perspectives on open problems," *arXiv preprint arXiv:2005.01643*, 2020.

[6] Ubisoft, "Trackmania," 2020. [Online]. Available: https://www.ubisoft.com/en-us/game/trackmania/trackmania

[7] A. Juliani, V.-P. Berges, E. Teng, A. Cohen, J. Harper, C. Elion, C. Goy, Y. Gao, H. Henry, M. Mattar *et al.*, "Unity: A general platform for intelligent agents," *arXiv preprint arXiv:1809.02627*, 2018.

[8] I. Kostrikov, A. Nair, and S. Levine, "Offline reinforcement learning with implicit q-learning," *arXiv preprint arXiv:2110.06169*, 2021.

[9] S. Fujimoto and S. S. Gu, "A minimalist approach to offline reinforcement learning," *Advances in neural information processing systems*, 2021.

[10] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative q-learning for offline reinforcement learning," *Advances in Neural Information Processing Systems*, 2020.

[11] A. Kobanda, C. Valliappan, J. Romoff, and L. Denoyer, "Learning computational efficient bots with costly features," in *IEEE Conference on Games (CoG)*, 2023.

[12] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, "D4rl: Datasets for deep data-driven reinforcement learning," *arXiv preprint arXiv:2004.07219*, 2020.

[13] A. Sestini, J. Bergdahl, K. Tollmar, A. D. Bagdanov, and L. Gisslén, "Towards informed design and validation assistance in computer games using imitation learning," in *IEEE Conference on Games (CoG)*, 2023.

[14] A. Nair, A. Gupta, M. Dalal, and S. Levine, "Awac: Accelerating online reinforcement learning with offline datasets," *arXiv preprint arXiv:2006.09359*, 2020.

[15] S. Lee, Y. Seo, K. Lee, P. Abbeel, and J. Shin, "Offline-to-online reinforcement learning via balanced replay and pessimistic q-ensemble," in *Conference on Robot Learning*. PMLR, 2022, pp. 1702–1712.

[16] M. Towers, J. K. Terry, A. Kwiatkowski, J. U. Balis, G. d. Cola, T. Deleu, M. Goulão, A. Kallinteris, A. KG *et al.*, "Gymnasium," Mar. 2023. [Online]. Available: https://zenodo.org/record/8127025

[17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[18] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International conference on machine learning*. PMLR, 2018.

[19] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International conference on machine learning*. PMLR, 2018.

[20] I. Uchendu, T. Xiao, Y. Lu, B. Zhu, M. Yan, J. Simon, M. Bennice, C. Fu, C. Ma, J. Jiao *et al.*, "Jump-start reinforcement learning," in *International Conference on Machine Learning*. PMLR, 2023.

[21] H. Zhang, W. Xu, and H. Yu, "Policy expansion for bridging offline-to-online reinforcement learning," *arXiv preprint arXiv:2302.00935*, 2023.

[22] G. Macaluso, A. Sestini, and A. D. Bagdanov, "Small dataset, big gains: Enhancing reinforcement learning by offline pre-training with model-based augmentation," in *Computer Sciences & Mathematics Forum*. MDPI, 2024.

[1]Env. and datasets are available at: https://github.com/ganjiro/OfflineMania

Authorized licensed use limited to: Comenius University. Downloaded on March 05,2025 at 09:17:18 UTC from IEEE Xplore. Restrictions apply.